

Proyecto 2: Divide y vencerás

Felipe Paillacar[†], Constanza Araya[†] y Guillermo Vargas[†]

[†]Universidad de Magallanes

14 de mayo de 2026

Resumen

Este proyecto aborda la implementación y el análisis experimental de algoritmos basados en la estrategia de divide y vencerás integrados a un sistema de gestión de deportistas. El estudio se centra en evaluar el rendimiento de técnicas de ordenamiento como Merge Sort (con optimización de umbral) y Quick Sort (con diversas variantes de pivote), junto con métodos de búsqueda binaria y exponencial. El trabajo integra un análisis de complejidad teórica contrastado con pruebas empíricas de tiempo de ejecución para determinar la eficiencia de cada algoritmo.

■ Índice

1	Objetivos	2
2	Introduccion	3
3	Conceptos teoricos	4
3.1	Divide y venceras	4
3.2	Complejidad de un algoritmo	4
	Conceptos clave de complejidad (notacion Big O)	
3.3	Esquema de particion de Lomuto	4
4	Algoritmos de ordenamiento	4
4.1	Merge Sort	5
4.2	Merge Sort con Insertion Sort	5
4.3	Quick Sort	5
5	Algoritmos de busqueda	7
6	Algoritmos de seleccion	8
7	Analisis de casos: ordenamiento	9
8	Analisis de casos: busqueda	10
9	Analisis de casos: seleccion	11
10	Conclusion	12

1. Objetivos

- Implementar correctamente algoritmos utilizando la estrategia de divide y vencerás.
- Analizar empíricamente la complejidad y el rendimiento de los algoritmos implementados, evaluando además el impacto de sus distintas optimizaciones y variantes.
- Documentar adecuadamente el código desarrollado y los resultados obtenidos del análisis.

2. Introduccion

3. Conceptos teoricos

3.1. Divide y venceras

Divide y venceras es una estrategia algoritmica. Un algoritmo divide y venceras tipico resuelve un problema siguiendo tres pasos:

- **Dividir:** Descompones el problema en subproblemas mas pequeños, cada subproblema debe representar una parte del problema original, Por lo general este paso emplea un enfoque recursivo para dividir el problema hasta que no es posible crear subproblemas mas pequeños.
- **Vencer:** Resolver los sub-problemas recursivamente. Esto se conoce como el caso base, es decir el subproblema es tan pequeño que se 'resolvio solo'.
- **Combinar:** Combinar apropiadamente las soluciones de los subproblemas para obtener la solucion del problema original.

3.2. Complejidad de un algoritmo

La complejidad es una medida de que tan eficiente es el algoritmo para resolver un problema, es decir es una medida de cuanto tiempo y espacio (memoria) requiere el algoritmo para producir una solucion. La complejidad de un algoritmo se expresa utilizando la notacion **Big O**. Esta notacion proporciona una forma de comparar la complejidad de distintos algoritmos. Por ejemplo, un algoritmo con una complejidad temporal de $O(n)$ se considera más eficiente que un algoritmo con una complejidad temporal de $O(n^2)$, porque crece a un ritmo más lento a medida que aumenta el tamaño de entrada. En general, el objetivo del diseño de algoritmos es encontrar algoritmos que sean eficientes y fáciles de implementar. Esto implica un compromiso entre la complejidad del tiempo y el espacio, así como otros factores como la simplicidad (facil entendimiento) y la mantenibilidad (facilidad de actualización, corrección o mejora).

3.2.1. Conceptos clave de complejidad (notacion Big O)

Es comun confundir la notacion Big O como una medida de tiempo, pero la realidad es que la notacion Big O nos indica cuantas operaciones debe realizar un algoritmo para resolver un problema, ya que si fuera una medida de tiempo podria variar dependiendo del hardware en el que se estan realizando las pruebas. Por ello se habla del mejor y peor caso, es decir el mejor caso o caso ideal seria cuando un algoritmo tiene que hacer operaciones minimas para llegar a la solucion. Por ejemplo, en el caso de la busqueda lineal, el mejor caso es cuando el elemento que se busca esta en la primera posicion del arreglo, por lo que solo se realiza una operacion. A continuacion se muestra una comparacion de las funciones de complejidad mas comunes.

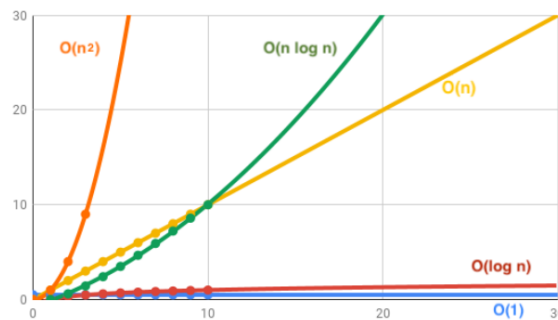


Figura 1. Comparacion de las funciones de complejidad mas comunes.

- $O(1)$: Tiempo constante. El tiempo de ejecución no cambia independiente del tamaño de la entrada.
- $O(\log n)$: Tiempo logaritmico, muy eficiente. El tiempo de ejecución crece lentamente a medida que aumenta el tamaño de la entrada.
- $O(n)$: Tiempo lineal. El tiempo de ejecución crece proporcionalmente al tamaño de la entrada.
- $O(n \log n)$: Tiempo log-lineal. Es casi una línea recta, pero con una ligera inclinación hacia arriba.
- $O(n^2)$: Tiempo cuadratico. El tiempo de ejecución crece proporcionalmente al cuadrado del tamaño de la entrada.

3.3. Esquema de particion de Lomuto

El esquema de particion de Lomuto es un algoritmo popular utilizado para dividir una matriz. Lomuto funciona seleccionando un elemento de pivote, generalmente el ultimo elemento del array, y luego dividiendolo en dos subarrays de modo que los elementos menores o iguales al pivote esten en el lado izquierdo del pivote y los elementos mayores queden a su derecha. El esquema de particion de Lomuto funciona de la siguiente manera:

- Se selecciona un elemento del array como pivote (generalmente el ultimo).
- Se recorre la lista comparando cada elemento con el pivote. Los menores se mueven a la izquierda y los mayores a la derecha.
- Al final del recorrido, se coloca el pivote en su lugar definitivo (entre las dos sublistas)
- Se repite el mismo proceso en la sublista izquierda y en la derecha hasta que todos los elementos estén ordenados.

4. Algoritmos de ordenamiento

Los algoritmos de ordenamiento son un conjunto de procedimientos que organizan los elementos de una lista o arreglo en un orden específico, ya sea ascendente o descendente. Los algoritmos de ordenamiento son importantes ya que facilitan la busqueda, la organizacion y el analisis de

datos. Son fundamentales en la eficiencia informatica ya que el desorden es costoso en terminos de tiempo y recursos. Algunas razones de su importancia incluyen:

■ Optimizacion de la busqueda

- **Busqueda lineal:** Si los datos estan desordenados, se debe revisar uno por uno hasta encontrar el elemento deseado (complejidad $O(n)$). En cambio, si los datos estan ordenados, se pueden utilizar algoritmos de busqueda mas efficientes.
- **Busqueda binaria:** Este algoritmo requiere que los datos esten ordenados para funcionar correctamente. Permite encontrar un elemento en tiempo logaritmico $O(\log n)$.

■ Organizacion y visualizacion:

 El ordenamiento permite agrupar elementos similares, lo cual es vital para:

- Eliminar duplicados: Es mas facil notar que un elemento esta duplicado si esta junto a su par.
- Priorizacion: En sistemas operativos o servidores, ordenar procesos por prioridad asegura que lo mas importante se ejecute primero.

Información

Antes de elegir un algoritmo de ordenamiento es importante hacer algunas preguntas: ¿Cuán grande son los datos que se ordenan? ¿Cuánta memoria hay disponible? ¿Los datos creceran?

4.1. Merge Sort

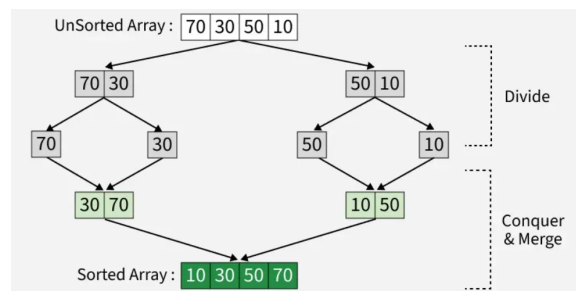


Figura 2. Merge sort no adaptativo

Merge sort es un algoritmo de ordenamiento conocido por su eficiencia y estabilidad. Utiliza la estrategia de divide y venceras para ordenar una lista o arreglo. Funciona dividiendo recursivamente el arreglo en mitades hasta que cada subarreglo contiene un solo elemento, luego combina esos subarreglos de manera ordenada para formar el arreglo final ordenado. Merge sort es un algoritmo no adaptativo, lo que significa que su rendimiento no mejora si el array esta previamente ordenado. Debido a esto su complejidad en el peor y mejor caso es $O(n \log n)$

4.2. Merge Sort con Insertion Sort

Como se menciona anteriormente en 4.1, merge sort es un algoritmo no adaptativo, lo que significa que su rendimiento no mejora si el array esta previamente ordenado. Para ello se puede optimizar utilizando insertion sort para ordenar subarreglos pequeños, ya que insertion sort es un algoritmo adaptativo, lo que significa que su rendimiento mejora si el array esta previamente ordenado, es decir Insertion sort no es 'ciego' como merge sort. Esto ayuda a que en teoria en el peor caso se mantenga la complejidad de merge sort $O(n \log n)$, pero en el mejor caso se mejora a $O(n)$, ya que insertion sort detecta que los subarreglos estan ordenados y como los subarreglos son pequeños, mejora significativamente el tiempo de ejecucion.

4.3. Quick Sort

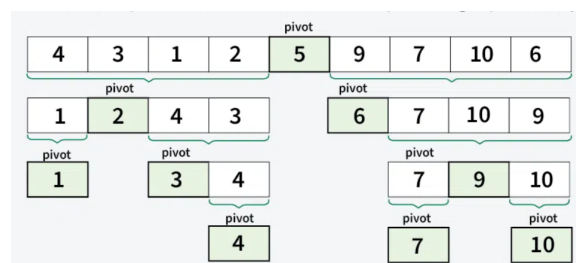


Figura 3. Quick sort

Quick sort es un algoritmo de ordenamiento que utiliza la estrategia de divide y venceras que elige un elemento como pivote y divide el array al rededor del pivote, como se nombro en 3.3. Este algoritmo utiliza recursividad, para ello es importante tener claro cual es el caso base, el cual es cuando el array tiene un solo elemento o esta vacio, ya que en ese caso el array ya esta ordenado. Hay diferentes formas de elegir un pivote, como por ejemplo:

- **Pivote primer elemento**
- **Pivote ultimo elemento**
- **Pivote aleatorio**
- **Pivote mediana de tres**

La eleccion del pivote es importante ya que puede afectar significativamente el rendimiento del algoritmo. Lo cual sera analizado en el transcurso de este proyecto.

5. Algoritmos de búsqueda

6. Algoritmos de seleccion

7. Analisis de casos: ordenamiento

8. Analisis de casos: busqueda

9. Analisis de casos: seleccion

10. Conclusion

```

1  /* Delete from a list */
2  /* Cell pointed to by P->Next is wiped out */
3  /* Assume that the position is legal */
4  /* Assume use of a header node */
5
6  void
7  Delete( ElementType X, List L )
8  {
9      Position P, TmpCell;
10
11     P = FindPrevious( X, L );
12
13     if( !IsLast( P, L ) ) /* Assumption of header use */
14     {                      /* X is found; delete it */
15         TmpCell = P->Next;
16         P->Next = TmpCell->Next; /* Bypass deleted cell */
17         free( TmpCell );
18     }
19 }

```

Código 1. Here is an implementation of a delete function for a singly linked list in C.([Source code](#))

■ Referencias

- [1] freeCodeCamp, *La complejidad de los algoritmos simples y las estructuras de datos en JS*, jul. de 2022. dirección: <https://www.freecodecamp.org/espanol/news/la-complejidad-de-los-algoritmos-simples-y-las-estructuras-de-datos-en-js/>.
- [2] T. S. Engineer, *Lomuto Partition Scheme*, sep. de 2023. dirección: <https://medium.com/@strivingengineer/lomuto-partition-scheme-f20eef44e76d>.
- [3] freeCodeCamp, *Significado del algoritmo divide y vencerás*, ene. de 2023. dirección: <https://www.freecodecamp.org/espanol/news/significado-del-algoritmo-divide-y-venceras/>.
- [4] E. Valley, *La complejidad de los algoritmos*, sep. de 2023. dirección: <https://europeanvalley.es/noticias/la-complejidad-de-los-algoritmos/>.
- [5] freeCodeCamp, *Explicación de la notación Big O con ejemplos de Python y JavaScript*, feb. de 2024. dirección: <https://www.freecodecamp.org/espanol/news/explicacion-de-la-notacion-big-o-con-ejemplo/>.
- [6] GeeksforGeeks, *Merge Sort*, mayo de 2024. dirección: <https://www.geeksforgeeks.org/dsa/merge-sort/>.
- [7] GeeksforGeeks, *Quick Sort Algorithm*, mayo de 2024. dirección: <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>.